Attention 加速器 IP 设计与验证

数字组招生实践项目（大二）A 题 · 规模 S = 8、d = 4

SystemVerilog (IEEE 1800-2012) · Icarus Verilog 12.0 · 450 个仿真用例全部通过

任务范围与实现结果

按题目正式任务规模用 SystemVerilog 实现基础 Attention RTL IP：S = 8 个 Token、d = 4 维，输入输出 8-bit 有符号整数。

必须先求 x_max，再计算 exp(score − x_max)
weight 为无符号定点，小数固定 4 位
输出 O = round(ACC / L)，8-bit 有符号

169

个时钟周期 / 次任务（100 MHz 下 1.69 μs）

450 / 450

仿真用例与黄金模型逐位一致

824 bit

片上存储（寄存器与 Buffer）

算法与定点规则

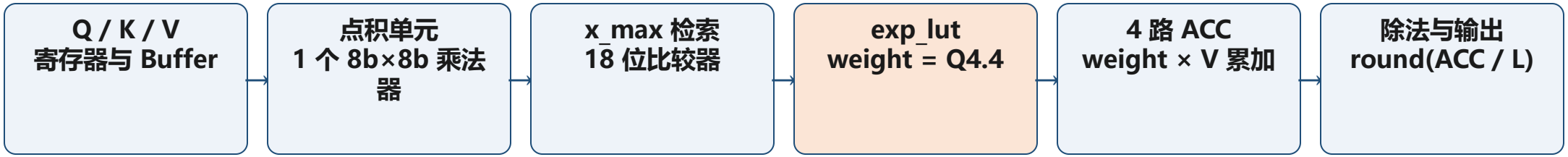
score_j = Σ Q[k] × K_j[k]
x_max = max(score_j)
weight_j = round(exp(score_j − x_max) × 16)
L = Σ weight_j
ACC[k] = Σ weight_j × V_j[k]
O[k] = round(ACC[k] / L)

题目手算示例核对（S = 2、d = 2）

Q = [1, 2]，K0 = [1, 0]，K1 = [0, 1]
V0 = [4, 2]，V1 = [1, 6]

score = [1, 2]，x_max = 2
weight = [6, 16]（0.375 与 1.000）
L = 22，ACC = [40, 108]
O = [2, 5]，与题目结果一致

硬件架构与数据流



Score Buffer 8 × 18b · Weight Buffer 8 × 5b · 4 路 ACC 16b · 状态机按 11 个状态调度数据流

单乘法器串行点积（4 拍 1 个 Score）控制面积；加权累加用 4 路并行乘加，8 拍完成 8 个 Token。

状态机与周期分解

状态	周期	主要动作
LOAD_Q	4	装载 Q 到寄存器
LOAD_KV	32	K/V 写入 Buffer
DOT	32	串行点积，得 8 个 Score
MAX	8	检索 x_max
EXP	8	查表得 weight，累加 L
ACC	8	4 路并行加权累加
NORM	72	4 次顺序除法
OUT	4	输出 O[0]~O[3]

合计 169 拍

100 MHz 下 1.69 µs / 次任务；done 之后立即回到 IDLE，可背靠背启动下一个 Query。

归一化占 43% 时间，原因是顺序除法器被 4 个输出分时复用。

指数查找表与归一化舍入

exp_lut 表内容（weight 为无符号 Q4.4）

mag	0	1	2	3	≥ 4
exp(−mag)	1.0000	0.3679	0.1353	0.0498	< 0.02
weight	16	6	2	1	0

Score 为整数，因此表只需覆盖 0~4 共 5 个地址；分数相差 5 以上的 Token 权重下溢为 0。

归一化舍入

除法器先取 |ACC|，加上 L/2 后整除 L，再恢复符号。

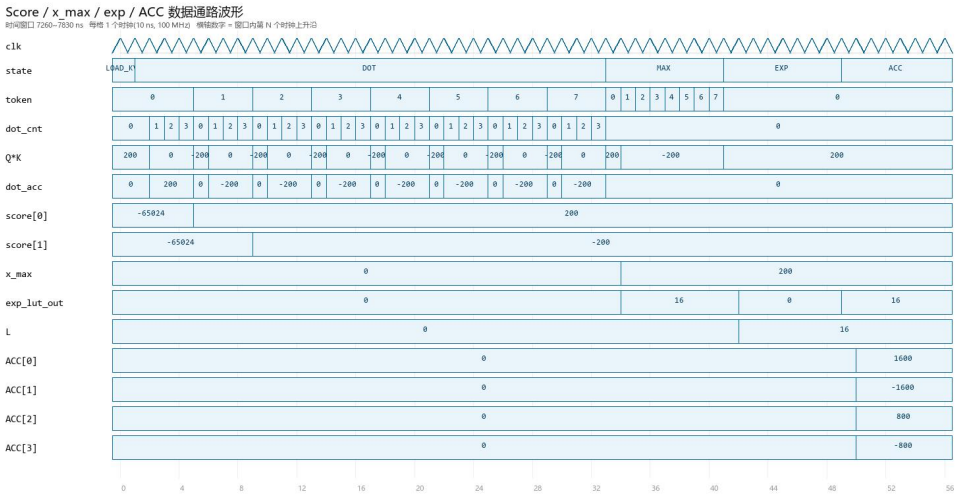
当 L 为偶数时 L/2 恰好是 0.5L；L 为奇数时等式 2|ACC| = L(2n+1) 左边偶数、右边奇数，不可能成立，因此不会出现半整数平局。

该实现与“四舍五入、0.5 远离 0”完全等价，一次除法 16 拍。

验证方法与结果

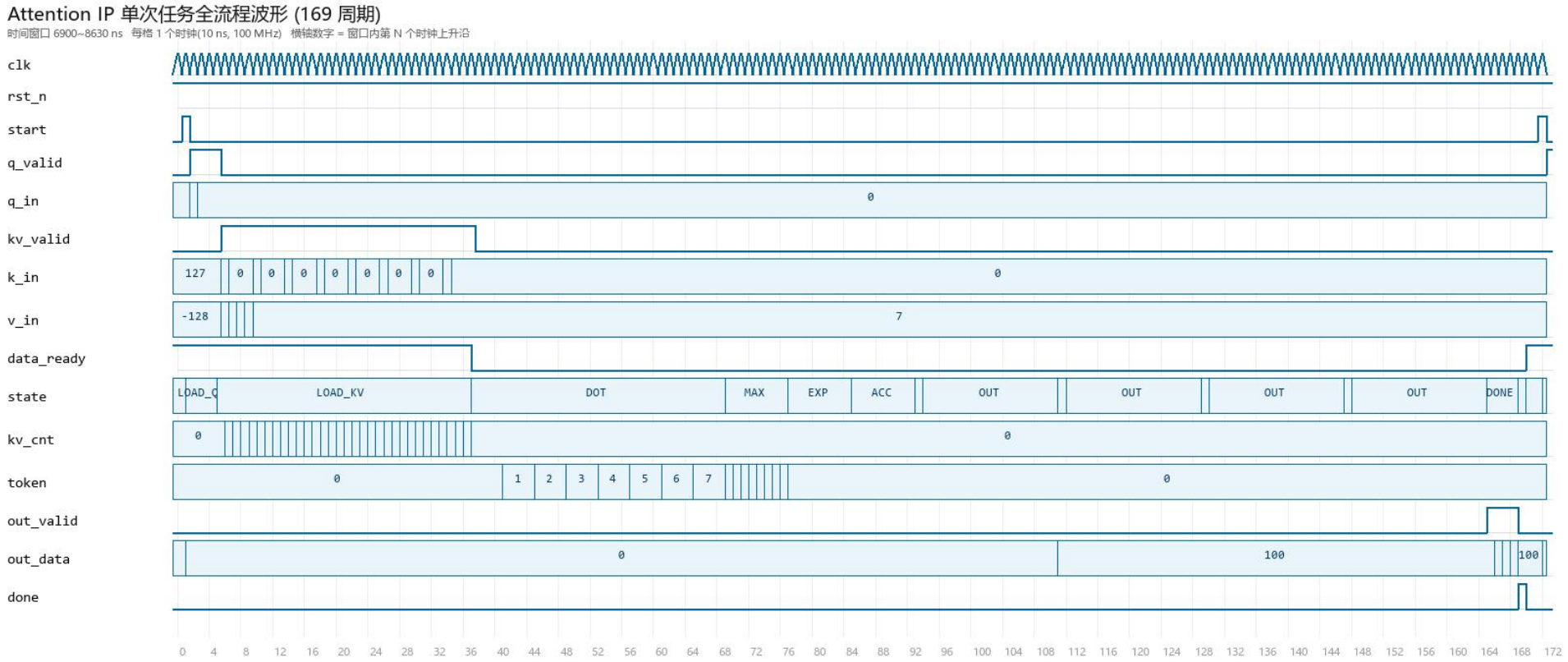
Python 黄金模型按题目规则生成期望值，testbench 驱动协议并逐项比对 4 拍输出，同时检查 done 时序与 X 值。

用例类型	数量	结果
定向用例	9	全部通过
随机用例	32	全部通过
压力测试	409	全部通过



随机覆盖全量程 ±128 与小量程 ±8；压力测试 400 个随机用例（种子 12345）。

单次任务波形：169 拍全流程



start 之后依次为 4 拍 Q、32 拍 K/V、点积、x_max 检索、指数查表、加权累加、4 次除法、4 拍输出，最后 done 拉高 1 拍

性能与序列长度扩展

项目	S = 8（本设计）	S = 256	变化
点积乘加次数	8 × 4 = 32	256 × 4 = 1024	32 倍
加权乘加次数	8 × 4 = 32	256 × 4 = 1024	32 倍
K / V Buffer	合计 512 bit（64 B）	合计 16 kbit（2 KB）	32 倍
片上存储合计	824 bit	约 26 kbit	32 倍

S = 256 时每个 Query 都要搬运 2 KB 的 K/V，带宽成为主要瓶颈；常见做法是批量复用 K/V、块状在线 Softmax、压缩搬运格式。

改进方向与结论

点积改 4 路并行或边装载边计算：装载与点积时间可压缩一半以上，代价是 4 个乘法器
归一化换倒数查找表或并行除法：从 72 拍降到 4~20 拍，代价是 ROM 或多套除法逻辑
weight 提升到 Q1.15 或用分段线性 / CORDIC：缓解权重下溢与最大 9 的输出偏差

结论

IP 已满足题目全部基础要求：x_max 与 exp(score − x_max) 均已实现，位宽逐级推导，450 个用例与黄金模型逐位一致。当前短板是 4 位小数权重在软 Softmax 场景下的量化误差，可通过提高精度或并行流水结构继续优化。

加分项实现与验证（A / B / C）

加分项	做法	实测结果
A 因果 Mask	新增 pos_i 输入与 MASK_EN 参数：j > pos_i 的 Key 不参与 x_max、权重置 0、不进 L 与 ACC	独立测试 tb_attention_mask.sv：pos_i = 0/3/7 三组全过（O = [10,0,0,0] / [35,10,0,0] / [58,19,0,0]）
B 并行点积	新增 DOT_PAR 参数：D 个乘法器一拍算完一个 Score（点积 32 拍 → 8 拍）	周期 169 → 145 拍（−14%），41 例 + 409 例压力测试全过；代价 +3 个 8×8 乘法器
C 参数化	S / D / DW / W_FRAC 全参数化，计数器位宽由 $clog2 自动推导	S 改为 4：41/41 通过，周期 169 → 125 拍；归一化占比升到 58%

基础版默认参数不变（MASK_EN=0、DOT_PAR=1），基础功能零回归；41 + 409 例仍全部通过、169 拍。

复现：run_sim.ps1 -Mask（因果 Mask 独立测试）· run_sim.ps1 -DotPar 4（并行点积）· 改 S 后 run_sim.ps1（参数化）